

Overview

This is the main project for the course. You will work in teams to deliver a substantial piece of NLP work that demonstrates technical competence, critical thinking, and original analysis. The work culminates in a 20-minute presentation in class followed by 5 minutes of Q&A.

Three options are available. They look different on the surface but share the same underlying expectation: produce evidence and analysis that are genuinely your own, grounded in primary work that you carried out as a team. Each option is structured so that personal judgment and hands-on work are required at every stage.

The three options at a glance

Choose one. They are equivalent in expected effort and weight.

Option	What you build	Best fit for
1. Application Development	A functioning NLP-powered application with a custom evaluation harness and real user feedback.	Teams that want to ship something concrete.
2. System Audit	A structured audit of a deployed NLP system, including a custom probe set and a failure taxonomy.	Teams that want to deeply understand existing systems.
3. Replication & Extension	A reproduction of a recent paper plus your own extension experiment.	Teams that want a research-flavored project.

Theme selection and approval

Before you start, your group must finalize the project topic in consultation with me. This step ensures that the project is well-aligned with course objectives, properly scoped, and genuinely interesting for the team.

Consider the following when choosing:

- **Relevance:** connect to current trends and real-world applications in NLP.
- **Feasibility:** ambitious but completable within the time available. Think big, start small.
- **Originality:** aim for a new angle, not a replica of a lab exercise.

- **Team commitment:** every team member should agree on and be invested in the chosen direction.

Once your group has a tentative topic, discuss it with me for approval before committing.

Option 1: Application Development

Build a functioning NLP-powered application that addresses a real-world use case. The system does not need to be production-grade; a clear proof of concept that exhibits the key envisioned functionalities is sufficient. Features that remain unrealized should be presented as future development directions grounded in what you have built.

Required components

- **Justification of need.** A clear argument for why the application is needed and where it would be applied. Identify existing approaches and articulate what your system brings that others lack.
- **Field review.** A concise but substantive review of the field or industry where your system would operate: trends, challenges, major players, existing solutions.
- **Functional system.** A working demo or POC that exhibits the core envisioned functionalities.
- **Custom evaluation.** Build your own test set of at least 30–50 representative examples with expected outputs and report quantitative results (accuracy, F1, BLEU, or whatever fits your task). Off-the-shelf benchmarks alone are not sufficient.
- **Failure mode analysis.** Document at least 5–10 cases where the system fails or behaves unexpectedly, with hypotheses about why.
- **Reproducible repository.** Clean code with a clear README and requirements file.

Practical advice: think big, start small. Aspire to a substantial problem (an AI doctor is more ambitious than a POS tagger), but commit to a modest, demonstrable POC. The exercise focuses on technical competence in NLP, not on marketability.

Option 2: System Audit

Pick a real, deployed NLP system (a chatbot, translation service, content moderator, voice assistant, search system, recommendation engine) and conduct a structured audit of its behavior.

The goal is to produce a document that someone considering deploying this system in their own context could use to make an informed decision.

This option does not require you to build a system, but it does require you to build substantial new evidence about an existing one. A pure descriptive review of how the system works will not meet the bar.

Required components

- **System characterization.** What the system does, its operations and features, and where it sits within a categorization of similar technologies.
- **Custom probe set.** Design and build your own test set of approximately 150–300 items, structured around hypotheses about where the system is likely to underperform: edge cases, bias, robustness, compositionality, multilingual handling, etc. Document the rationale for your test design.
- **Empirical findings.** Run your probes against the system, document outputs with evidence (screenshots, transcripts, logs), and quantify failure rates by category.
- **Failure taxonomy.** Organize observed failures into a coherent taxonomy with clear definitions and representative examples.
- **Comparison.** Evaluate at least one comparable alternative system on the same probe set and compare results.
- **Technical context.** Discuss the underlying technologies, algorithms, and methods relevant to systems of this type, drawing from academic literature. Connect your empirical findings to known challenges in the literature.
- **Mitigations and future directions.** Based on your findings and the literature, propose specific mitigations for the failures you identified and outline plausible directions for future iterations.

Practical advice: choose a system you can probe extensively. APIs with restrictive rate limits or paywalls will become a bottleneck. Open-weight models hosted locally, free public APIs, or systems with generous trial tiers tend to work better.

Option 3: Replication & Extension Study

Pick a recent NLP paper (preferably from the last three years, from a reputable venue, with publicly available code) and conduct a replication-and-extension study. The objective is not to summarize the field but to produce primary evidence of your own that contributes to a critical understanding of one specific contribution.

Required components

- **Paper selection.** Justify the choice of paper. Why is it interesting? What does it claim? How influential or representative is it?
- **Replication.** Reproduce the main result reported in the paper. Document the process, including any deviations, undocumented assumptions, or implementation issues encountered. Compare your reproduced numbers against the published ones and discuss any gaps.
- **Extension experiment.** Design and execute one extension that goes beyond the original paper: applying the method to a different language, a different domain, a smaller or larger model, a different evaluation set, or adding an ablation the authors did not perform. The extension must be substantive: a one-line change is not sufficient.
- **Mini-survey, anchored in the paper.** A focused survey of the surrounding literature with the chosen paper as the anchor. The survey should help readers understand where the paper fits, what came before, and what came after.
- **Critical assessment.** Based on your replication and extension, what holds up? What is brittle? What questions remain open?
- **Open question.** Identify a concrete open question or research direction grounded in what you observed empirically.

Practical advice: choose a paper whose code actually runs. Available code matters more than the paper's prestige. Skim the GitHub issues before committing. Many published repositories are abandoned or under-documented.

Recommended workflow

The following pacing is a suggestion, not a requirement. Adjust to your team's circumstances and to the specifics of your project.

- **Early phase:** finalize the topic and scope, agree on roles, and do an initial scan of the literature or the system landscape.
- **Middle phase:** core empirical work (build, audit, or replicate, depending on option).
- **Late phase:** analysis, writing, slide preparation, and an internal rehearsal of the presentation.

Teams that pace their work and surface problems early tend to deliver substantially stronger projects, but the structure of your team's work is your decision.

Evaluation criteria

The following rubric applies across all three options.

- **Technical execution and rigor.** Correctness of methodology, soundness of experimental design, quality of code and artifacts.
- **Empirical evidence and depth of analysis.** The extent to which conclusions are grounded in your own data, observations, and measurements rather than restated from external sources.
- **Originality and critical thinking.** How far the work goes beyond the obvious and surfaces non-trivial insights.
- **Communication.** Clarity, structure, and quality of both the written report and the in-class presentation.
- **Reproducibility and quality of deliverables.** Completeness of code, datasets, prompts, and supporting documentation. A reader should be able to follow and reproduce your work.

Use of LLMs and AI assistance

You may and probably should use LLMs throughout this project : to brainstorm, debug, draft prose, explore the literature, write boilerplate code, summarize papers. This reflects how applied NLP is actually practiced today.

That said:

- The substantive intellectual decisions like problem framing, methodology, test set design, label definitions, error interpretations, conclusions must come from your team.
- Include a short section in the final report titled “Use of AI tools” describing how you used LLMs in the project.
- During Q&A, members of the team should be able to defend any choice in the work. If a team member cannot explain why something was done a certain way, that will weigh against the project.

Individual accountability within the group

Group projects can mask significant differences in individual contribution. To keep things fair:

- Each team member submits a brief individual reflection (about one page) at the end of the project, describing specific contributions, what was learned.
- I reserve the right to differentiate individual grades from the team grade where contributions are clearly uneven.

Submission guidelines

All teams submit the following via the course platform:

- **Technical report (PDF).** Length appropriate to the project.
- **One-page executive summary.** Written for a non-technical reader.
- **Slides** used during the in-class presentation.
- **Code repository (link).** For Option 1, the full system. For Option 2, the probe set and analysis scripts. For Option 3, the replication code and the extension code.
- **Supporting artifacts.** Dataset, probe set, prompts, or any other material referenced in the report.

- **Individual reflections**, one per team member.
- **For Option 1 only:** a user manual and an installation/execution guide. If you can host the application online, include the link.

Research repositories

- **Google Scholar:** broad academic search. <https://scholar.google.es/>
- **arXiv:** open-access pre-prints, widely used in NLP/ML. <https://arxiv.org/>
- **ACL Anthology:** the canonical NLP repository, free access. Focus on conference papers from 2015 onwards. <https://aclanthology.org/>
- **IEEE Xplore:** engineering and CS literature, partial open access. <https://ieeexplore.ieee.org/>
- **Semantic Scholar:** AI-powered scientific search. <https://www.semanticscholar.org/>
- **Papers with Code:** papers paired with implementations and benchmarks; especially useful for Option 3. <https://paperswithcode.com/>

Final note

The strongest projects in this kind of assignment are not necessarily the most technically ambitious. They are the ones whose authors can articulate, with concrete observations from their own work, what they learned and why it matters. Aim for that.